

AI Puzzle Solver - Project Report

-Ram Raghuwanshi

25BAI10166

1. Abstract

My project focuses on solving the classic 8-puzzle (3 by 3) problem through an intelligent, AI-driven approach. I implemented the A* search algorithm alongside the Manhattan Distance heuristic, so that my system does not just find a solution, it finds the most efficient one. To make these hidden calculations visible, I developed a graphical interface that allows users to actually see every step the AI took to solve the puzzle.

2. Introduction: Why the 8-Puzzle ?

The 8-puzzle is a foundational challenge in the world of AI, consisting of 3 by 3 grid where eight numbered tiles must be slid into a specific order. For a human, it's a game of patience; for a computer, it's a test of search efficiency. I developed this project to move beyond "brute-force" methods—where a computer tries every possible move—and instead implement

informed search strategies that allow the machine to "reason" about which move brings it closer to the finish line.

3. Methodology

3.1 A* Search Algorithm

The A* algorithm serves as the "brain" of the solver, it constantly evaluates the best move using the formula:

$$f(n) = g(n) + h(n)$$

- **$g(n)$** represents the cost/moves made to reach the current state.
- **$h(n)$** is the estimated cost remaining to reach the goal. By balancing past effort with future potential, the algorithm ensures it always picks the shortest path.

3.2 Manhattan Distance Heuristic

As it is obviously noticeable, along with A* search algorithm, I chose the Manhattan Distance heuristic. This helps in calculating how many vertical and horizontal steps each tile is away from its "home" position. I selected this method because it is simple to compute yet powerful enough to guarantee that the AI finds an optimal solution every single time.

4. System Architecture

The software I built, is like a partnership between two specialized modules:

- **The Solver Logic (solver.py):** You can compare this to the engine room where the A* algorithm lives. There it generates the sequence of moves required to solve the board.
- **The Graphical Interface (main.py):** You can say this as "face" of the project, which I built with Python's Tkinter library. It lets users interact with tiles, use the solver, and even see the path the solver took.

5. Challenges Faced

Managing Building an AI from scratch came with its share of technical "puzzles":

- **Efficiency:** I had to find ways to represent the board state so the computer wouldn't slow down. This was not very easy and I had many issues while trying to improve efficiency.
- **Loop Prevention:** I had to make sure the AI did not get stuck repeating the same moves again and again. This did take a long time but also taught me a lot.

- **Design:** I had to craft an interface that felt intuitive and responsive to the user. I had to learn tkinter module for this and it definitely was not as easy as it sounded.

6. Limitations

While effective, my current version has a few boundaries:

- The logic is currently locked to the 3 by 3 grid format.
- If the puzzle is extremely scrambled, the computer's memory usage can spike.
- Sometimes, the screen might briefly "freeze" while the AI is doing the heavy lifting for a complex solve.

7. Conclusion

This project has been a rewarding way to actually see how AI or specifically saying how A* search algorithm actually works. The A* algorithm proved to be an incredibly efficient tool for solving the puzzle optimally. By adding the GUI, I was able to bridge the gap between complex code and a visual experience that makes the "thinking" process of the machine easy for anyone to understand.

8. Future Improvements

Looking ahead, I hope to push my project even further:

- I will be trying to add **animations** to make the tile transitions look very smooth.
- I will try to expand the code to handle even larger grids 4 by 4, 5 by 5 and even n by n grids where user gives n .
- Implementing a "battle mode" where user can compare the speed of A* against other algorithms like BFS or DFS. This will further improve visualisation of these algorithms and even help to understand how each of these works.